
Route to Reason: Adaptive Routing for LLM and Reasoning Strategy Selection

Zhihong Pan, Kai Zhang, Yuze Zhao, Yupeng Han
 State Key Laboratory of Cognitive Intelligence
 University of Science and Technology of China
 {panzh, yuzezhao, yupenghan}@mail.ustc.edu.cn
 kkzhang08@ustc.edu.cn

Abstract

The inherent capabilities of a language model (LM) and the reasoning strategies it employs jointly determine its performance in reasoning tasks. While test-time scaling is regarded as an effective approach to tackling complex reasoning tasks, it incurs substantial computational costs and often leads to "overthinking", where models become trapped in "thought pitfalls". To address this challenge, we propose Route-To-Reason (RTR), a novel unified routing framework that dynamically allocates both LMs and reasoning strategies according to task difficulty under budget constraints. RTR learns compressed representations of both expert models and reasoning strategies, enabling their joint and adaptive selection at inference time. This method is low-cost, highly flexible, and can be seamlessly extended to arbitrary black-box or white-box models and strategies, achieving true plug-and-play functionality. Extensive experiments across seven open source models and four reasoning strategies demonstrate that RTR achieves an optimal trade-off between accuracy and computational efficiency among all baselines, achieving higher accuracy than the best single model while reducing token usage by over 60%. The code is available at <https://github.com/goodmanpzh/Route-To-Reason>.

1 Introduction

With the continuous advancement of large language models (LLMs), their generality and autonomy have demonstrated human-like or even superhuman capabilities. In this context, reasoning ability has undoubtedly become the core driver of intelligent agent behavior [1]. Consequently, an increasing number of reasoning models [2–4] and reasoning strategies [5–11] have emerged. These expert models and reasoning strategies synergize and evolve, collectively pushing the boundaries of language models' reasoning capabilities.

This raises a critical question worthy of in-depth exploration: Given such a rich selection space of expert models and reasoning strategies, *how can we efficiently identify the most suitable pairing within their combinatorial space?*

Intuitively, one might prefer combining powerful reasoning models (e.g., o3 [12]) with sophisticated reasoning strategies (e.g., Chain-of-Thought [13]) to tackle complex problems. Particularly under the guidance of the test-time scaling paradigm, allocating a high budget to enhance performance appears to be a natural choice.

However, this intuition-driven, fixed pairing approach may face two key challenges in practice: **Performance bottlenecks:** Existing research [14–21] suggests that "overthinking" can trap the reasoning process in protracted local reasoning patterns, limiting the model's ability to deviate from the current reasoning path, thereby degrading performance. **Budget inefficiency:** For low-difficulty tasks, employing high-performance models and complex strategies not only fails to yield significant

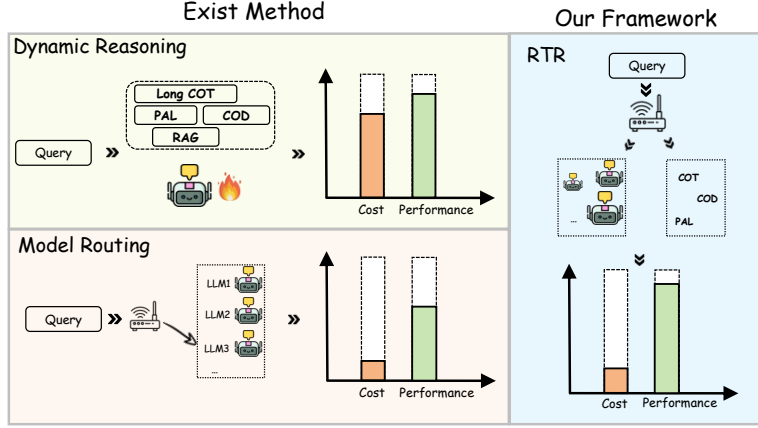


Figure 1: We propose Route-to-Reason (RTR), a low-cost and flexible expert selection framework capable of jointly optimizing model and strategy selection.

gains but also leads to resource waste. We believe “less is more”: lighter expert-strategy pairings often achieve better cost-performance trade-offs.

Some prior explorations have focused on model routing [22–31], enabling the system to select the most suitable model from a pool based on the input. However, most existing routing methods overlook the intricate relationship among expert model performance, reasoning strategies, and input complexity, often resulting in suboptimal decisions. While the works of [16, 32–36] approach the problem from the perspective of dynamic reasoning strategy selection. These approaches, with the goal of tailoring the reasoning process to input characteristics, enhance performance and enable dynamic scaling at test time. Substantial performance variation across expert-strategy combinations and input difficulties remains underexplored. A principled approach to modeling these differences and selecting models accordingly could further improve performance and efficiency.

To address this, we propose a unified framework for joint model and strategy routing, enabling efficient and accurate test-time computation through dynamic selection. Specifically, we represent each expert and each reasoning strategy using learnable vector that capture their respective performance and computational cost characteristics. Given an input instance, we encode the query using a pretrained LM. Then, we design two modules to predict the expected performance and response tokens for all model-strategy combinations, thereby constructing a routing table. Based on this table, a routing policy selects the optimal model-strategy pair that maximizes efficiency while improving accuracy.

Compared to previous approaches, our framework dynamically adapts to questions of varying difficulty and intelligently selects the most appropriate model-strategy pair for each query. This leads to a more optimal trade-off between computational cost and reasoning performance. The key distinctions between our approach and existing methods are illustrated in Figure 1. Existing methods often fail to achieve an optimal balance between cost and performance. In contrast, by jointly selecting both the model and the reasoning strategy, our framework achieves superior performance at a reduced computational cost.

We conduct experiments on seven challenging reasoning tasks (language understanding, scientific reasoning and mathematical reasoning) to evaluate the proposed RTR in both in-distribution and out-of-distribution settings. Results show that our approach consistently improves reasoning accuracy while reducing the average number of generated tokens by over 60% compared with single best model, validating its effectiveness.

2 Methodology

2.1 Motivation

Recently, reasoning-enhanced language models have demonstrated strong performance on complex tasks by leveraging extended reasoning steps and structured thinking [3, 4]. However, their advantages tend to diminish on simpler problems. As illustrated in Figure 2, while the thinking model (QwQ)

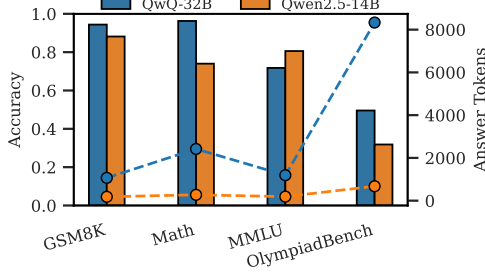


Figure 2: Performance and average answer tokens distribution of two different LLMs when responding to queries from subsets of four reasoning tasks.

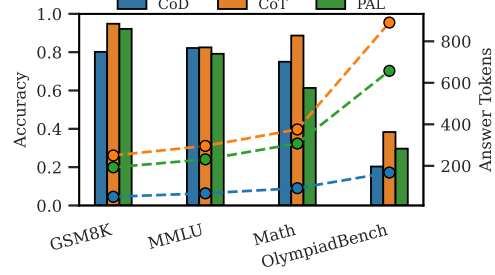


Figure 3: Performance and average answer tokens distribution of Qwen2.5-14B-Instruct under different reasoning strategies when responding to queries from subsets of four reasoning tasks.

shows substantial improvements over the non-thinking model (Qwen2.5-14B-Instruct) on complex tasks such as Math [37] and OlympiadBench [38], it offers only marginal improvements or even slight performance drops on relatively straightforward and commonsense tasks like GSM8K [11] and MMLU [39], despite incurring significantly higher inference costs, often generating up to 10 times more tokens. This highlights a critical challenge: rather than universally deploying heavy reasoning models, it becomes essential to determine when such complex reasoning is truly necessary.

Moreover, we observe that reasoning strategies play as crucial a role as model selection. As shown in Figure 3, strategies such as CoD [8] prompting can significantly reduce answer length while achieving performance comparable to CoT [13] prompting on easier tasks. Additionally, PAL [7] performs well on arithmetic-intensive tasks and generates more consistent outputs. These observations motivate our approach: jointly selecting both the model and the reasoning strategy for each input query enables adaptive, cost-effective inference while maintaining strong overall performance.

2.2 Problem Formulation and Preliminaries

We consider a collection of language models $\mathcal{M} = \{m_j : j = 1, \dots, M\}$, each differing in size or capability, and a set of reasoning strategies $\mathcal{S} = \{s_k : k = 1, \dots, K\}$. Given a set of input queries $\mathcal{D} = \{x_i : i = 1, \dots, N\}$, applying model m_j with strategy s_k to query x_i yields a response characterized by two quantities: the performance score $a_{i,j,k}$ (e.g., accuracy, utility, or another task-specific metric), and the number of generated tokens $l_{i,j,k}$ serving as a proxy for inference cost. Our objective is to predict these two quantities and select an appropriate model-strategy pair (m_j, s_k) based on these quantities for each query x_i . Formally, we seek to learn a routing function:

$$\pi : \mathcal{X} \rightarrow \mathcal{M} \times \mathcal{S},$$

where $\pi(x_i) = (j^*, k^*)$ denotes the selected model and strategy for x_i . The objective is to optimize the trade-off between total performance and total generation cost:

$$\max_{\pi} \sum_{i=1}^n a_{i,j^*,k^*} - \lambda \sum_{i=1}^n l_{i,j^*,k^*},$$

where $\lambda > 0$ is a hyperparameter that balances performance and efficiency, a_{i,j^*,k^*} and l_{i,j^*,k^*} denote the performance and generated length, respectively.

2.3 RTR Framework

Method Overview. As shown in Figure 4, for each input question, we first encode it into a dense vector representation using a pretrained encoder. Additionally, each candidate model and reasoning strategy is represented by an embedding that captures its characteristics, such as performance capability and computational cost. These embeddings, together with the question representation, are concatenated and passed into two predictors: one estimates the expected performance, and the other predicts the expected answer length. This yields a routing table for each question containing the predicted performance and token usage for every model-strategy pair. Finally, a routing policy, controlled by a trade-off coefficient between performance and cost, selects the optimal model

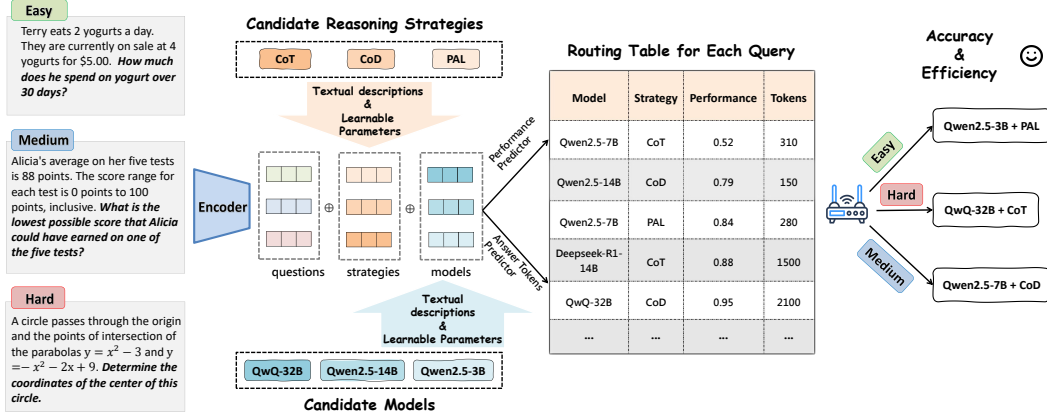


Figure 4: The RTR first encodes the input question, available models, and reasoning strategies. Two predictor modules then estimate the expected performance and token usage for each model-strategy combination, generating a routing table. Finally, the router selects the most suitable model-strategy pair that balances accuracy and efficiency for each question.

and strategy combination for each query. The training and inference procedures are provided in Algorithm 1.

Model and Strategy Representation. The prediction performance for each model-strategy pair critically depends on how well their characteristics are represented. To this end, we design a dual-component representation for each model and strategy. The first component is obtained by encoding a textual description of the model or strategy using a pretrained encoder, similar to how we encode input questions. These descriptions are generated using an auxiliary LLM aiming to capture high-level semantic attributes such as capability and usage scenarios. The detailed textual descriptions used for models and strategies are provided in Appendix B.3. This component is fixed during training. The second component is a randomly initialized embedding that is learned and updated during training. This learnable component enables the model to capture fine-grained behavioral nuances and task-specific adaptation signals that may not be easily conveyed through textual descriptions. These two components combine external knowledge and internal task-specific patterns to form a comprehensive representation.

Dual Prediction Module. After obtaining the representations of the query, model, and strategy, we construct two predictor modules to estimate the response quality and output token usage of each model-strategy combination for a given query. Specifically, we use multi-layer perceptron (MLP) for each prediction task. These predictors are trained independently using their respective loss functions.

The performance prediction module estimates the likelihood that a given model-strategy pair will correctly answer the input query. Since the correctness labels in the training data are binary, we use a binary cross-entropy loss:

$$\hat{a}_{i,j,k} = \text{MLP}_{\text{perf}}(z_{i,j,k}), \quad \mathcal{L}_{\text{perf}} = -y \log(\hat{a}_{i,j,k}) - (1 - y) \log(1 - \hat{a}_{i,j,k}), \quad (1)$$

where $z_{i,j,k}$ denotes the input to the performance predictor for query i and model-strategy pair (j, k) , and y is the ground-truth correctness label.

The token usage prediction module estimates the expected number of output tokens required to generate a response. As this is a regression task, we adopt mean squared error (MSE) loss:

$$\hat{l}_{i,j,k} = \text{MLP}_{\text{len}}(z_{i,j,k}), \quad \mathcal{L}_{\text{len}} = (\hat{l}_{i,j,k} - l_{i,j,k})^2, \quad (2)$$

where $z_{i,j,k}$ denotes the input to the token usage predictor, and $l_{i,j,k}$ is the ground-truth response length.

Routing Table and Routing Policy. Inspired by routing concepts in computer networks, we design a routing mechanism based on a predicted performance-cost table. For each query, we generate a routing table that organizes the predicted performance and output tokens for all combinations

of candidate models and strategies as shown in Figure 4. This table encapsulates the necessary decision-making information, akin to a routing table in network systems, and serves as the foundation for downstream routing decisions.

To determine the best routing destination for a given query, we propose a simple yet effective scoring function. The final routing score $s_{n,l,s}$ for query q_n routed to model l with strategy s is computed as a weighted combination of predicted performance and cost:

$$\text{score}_{i,j,k} = \lambda \cdot \hat{a}_{i,j,k} - (1 - \lambda) \cdot \hat{l}_{i,j,k}. \quad (3)$$

Here, $\hat{a}_{i,j,k}$ and $\hat{l}_{i,j,k}$ denote the predicted performance score and predicted cost (e.g., based on output length) respectively. The parameter $\lambda \in [0, 1]$ controls the trade-off between quality and cost: a higher λ prioritizes performance (quality-first), while a lower λ encourages cost-efficiency. The optimal routing decision is then made by selecting the candidate with the highest score:

$$(j^*, k^*) = \arg \max_{j,k} \text{score}_{i,j,k}. \quad (4)$$

This approach offers flexibility in adapting to different application needs. For instance, by tuning λ , users can customize the system to prefer higher-quality responses, lower costs, or a balanced trade-off between the two. We do experiments to show its effectiveness.

3 Experiment

3.1 Experimental Setup

Candidate LLMs and Strategies. We choose six open-source LLMs from HuggingFace¹: Non-thinking models [3] include Qwen2.5-3B, Qwen2.5-7B, and Qwen2.5-14B, which are general-purpose models designed for a wide range of tasks, with performance generally improving with model size. Thinking models [3, 4] include DeepSeek-R1-7B, DeepSeek-R1-14B, and QwQ-32B, which are optimized for reasoning tasks. These models are trained to produce longer reasoning steps for complex problems.

We evaluate four reasoning strategies: **Vanilla**, which uses the original question without any additional prompting and serves as a baseline; **CoT** [13] prompts the model to generate intermediate reasoning steps before answering; **PAL** [7] prompts the model to solve questions by generating executable code; and **CoD** [8] prompts the model to generate only intermediate drafts with explicit constraints on output length, encouraging concise reasoning. Detailed prompts for each strategy are provided in Appendix B.2.

Datasets. The statistics of the datasets used are summarized in Table 1. We select subsets from four reasoning tasks of varying difficulty: (i) GSM8K [11], a mathematical reasoning dataset with diverse grade school word problems; (ii) MMLU [39], a general-purpose benchmark, where we evaluate on a selected subset of STEM subjects; (iii) Math [37], a large-scale dataset focusing on diverse math problems requiring numerical reasoning and problem solving; and (iv) OlympiadBench [38], a challenging benchmark derived from olympiad-level scientific problems. We construct our dataset by

Algorithm 1 Training and Inference of RTR.

Require: Training set $\mathcal{D}_{\text{train}}$, encoder E , descriptions $\{d_j, d_k\}$, trade-off λ

- 1: Initialize learnable embeddings $\{e_j\}, \{e_k\}$ and MLPs: $\text{MLP}_{\text{perf}}, \text{MLP}_{\text{len}}$
- 2: Encode descriptions: $z_j = [E(d_j); e_j]$, $z_k = [E(d_k); e_k]$
- 3: **for** each $(x_i, \{a_{ijk}, l_{ijk}\}_{j,k})$ in $\mathcal{D}_{\text{train}}$ **do**
- 4: $q_i = E(x_i)$
- 5: **for** each (j, k) **do**
- 6: $z_{i,j,k} = [q_i; z_j; z_k]$
- 7: Update $\text{MLP}_{\text{perf}}, \text{MLP}_{\text{len}}$ using BCE and MSE losses
- 8: **end for**
- 9: **end for**
- 10: **Inference:**
- 11: **for** each test x_n **do**
- 12: $q_n = E(x_n)$
- 13: **for** each (j, k) **do**
- 14: $z_{n,j,k} = [q_n; z_j; z_k]$
- 15: Predict $\hat{a}_{n,j,k}, \hat{l}_{n,j,k}$ via MLPs
- 16: $\text{score}_{n,j,k} = \lambda \hat{a}_{n,j,k} - (1 - \lambda) \hat{l}_{n,j,k}$
- 17: **end for**
- 18: Select $(j^*, k^*) = \arg \max_{j,k} \text{score}_{n,j,k}$
- 19: **end for**

¹<https://huggingface.co/>

collecting responses generated by candidate LLMs using different reasoning strategies, and split it into 70% for training and 30% for testing.

To assess generalization, we further evaluate the trained router on three out-of-distribution (OOD) benchmarks: (i) SciQ [40], a multiple-choice science question dataset for middle school level; (ii) PIQA [41], which tests physical commonsense reasoning through everyday scenarios; and (iii) ARC-C [42], a challenging subset of the ARC benchmark focusing on questions requiring multi-step reasoning and commonsense inference. These datasets are not included in the training phase and are used solely to evaluate the router’s out-of-distribution performance.

Baselines. We compare our proposed model against the following baselines. First, we consider two single-model baselines: **Qwen2.5-3B**: The smallest single LLM in our pool of candidate LLMs; **QwQ-32B**: The best single LLM in our pool of candidate LLMs.

Next, we compare our method with several routing-based baselines. The detailed settings of these baselines are provided in Appendix B.1. **Random**: Randomly selects a model-strategy pair from the pool of candidates; **KNN-Router** [28]: For a given test query, the KNN-router identifies the k nearest neighbors from the training set in the embedding space. For each candidate model-strategy pair, it estimates the expected performance by averaging the observed rewards over these neighbors, and selects the pair with the highest estimated score; **RouteLLM** [30]: A binary router that assigns queries to either a strong or weak model based on learned routing policies over query features; **EmbedLLM** [22]: Constructs an embedding for each model using matrix factorization to capture its performance profile across the dataset, and uses these embeddings to route queries to the most suitable LLM.

Evaluation metrics. Routing methods are evaluated using two primary metrics: *accuracy*, which measures the correctness of the selected model’s response on each task, and the *average number of tokens*, reflecting the efficiency in controlling computational cost.

Implementation Details. For non-reasoning models, outputs are generated via greedy decoding (temperature = 0). For reasoning-based models, we follow official recommendations, using temperature = 0.6 and top-p = 0.95 for sampling. For the router, textual descriptions of each model, reasoning strategy, and input query are encoded with the `all-mpnet-base-v2`² model from the sentence-transformers library, yielding 768-dimensional embeddings. Both predictors are implemented as MLPs with a hidden size of 768. Training is performed for up to 100 epochs using the Adam optimizer (batch size 32, initial learning rate 1×10^{-3}), with early stopping based on test loss.

3.2 Main Results

In-Distribution Results. Table 2 presents the performance comparison of various routing methods across four reasoning tasks. First, we observe that all routing models outperform random routing, demonstrating the effectiveness of employing routing strategies. Our proposed method, **RTR**, achieves the best overall accuracy (82.5%) while significantly reducing the average output length (1091.3 tokens). It outperforms all baselines in terms of the trade-off between performance and cost. Specifically, RTR achieves the highest accuracy on MMLU and OlympiadBench, and the second-best results on GSM8K and Math. Compared to the best-performing baseline, EmbedLLM, RTR matches or surpasses its accuracy while reducing the average token usage by over 39.6%. Notably, compared to the single largest model, QwQ-32B, which achieves strong performance at a very high cost, RTR improves average accuracy by 2.5 percentage points while reducing output token length by more than 60%. These results demonstrate that RTR can effectively select the most appropriate LLM and strategy to achieve both high accuracy and efficiency.

²<https://huggingface.co/sentence-transformers/all-mpnet-base-v2>

Table 2: Results on in-distribution datasets. The best router-based result is in **bold**, and the second-best is underlined.

Model	GSM8K		Math		MMLU		OlympiadBench		AVG	
	Accuracy	Tokens	Accuracy	Tokens	Accuracy	Tokens	Accuracy	Tokens	Accuracy	Tokens
Qwen2.5-3B	71.5	205.4	54.3	295.5	61.7	253.1	21.0	1007.2	56.0	371.7
QwQ	94.4	1148.5	95.6	2583.0	71.8	1219.3	48.5	8762.3	80.0	2745.2
random	84.4	382.4	77.7	<u>1043.7</u>	69.4	723.1	32.6	4219.1	69.5	1271.6
KNN-Router	89.2	272.9	88.3	1122.3	78.1	347.5	36.6	<u>4197.7</u>	76.9	<u>1101.3</u>
RouteLLM	91.0	372.0	89.3	1161.8	72.8	597.3	45.5	7696.8	77.3	1814.3
EmbedLLM	95.8	927.1	94.8	1898.2	80.5	508.8	41.5	5786.4	81.9	1808.3
RTR (Ours)	<u>95.2</u>	<u>297.8</u>	<u>92.9</u>	982.9	82.5	<u>432.5</u>	45.5	3399.7	82.5	1091.3

Table 3: Results on out-of-distribution datasets. The best is in **bold** and the second-best is underlined.

Model	PIQA		SciQ		ARC-C		AVG	
	Accuracy	Tokens	Accuracy	Tokens	Accuracy	Tokens	Accuracy	Tokens
Qwen2.5-3B	73.3	150.8	68.2	187.6	65.3	248.9	68.9	195.8
QwQ	95.5	1126.4	93.8	1203.6	91.7	1831.9	93.7	1387.3
random	84.6	350.0	78.3	430.8	74.1	713.9	79.0	498.2
KNN-Router	90.3	<u>277.4</u>	89.2	376.2	86.3	550.6	88.6	<u>419.4</u>
RouteLLM	93.2	363.1	<u>92.7</u>	894.3	91.2	1103.2	92.3	786.9
EmbedLLM	<u>95.1</u>	832.8	<u>92.2</u>	1002.3	<u>92.4</u>	1631.1	<u>93.2</u>	1155.4
RTR (Ours)	95.3	222.3	94.2	<u>405.7</u>	93.1	<u>553.7</u>	94.2	393.9

Out-Of-Distribution Results. As shown in Table 3, the proposed RTR achieves the highest average accuracy across all out-of-distribution datasets, surpassing the best-performing individual LLM (QwQ) by a substantial margin of 0.5%. Notably, RTR achieves this strong performance while significantly reducing the average number of response tokens (393.9 vs. 1387.3), indicating superior efficiency. Among all routing-based baselines, only RTR is able to outperform QwQ in terms of average accuracy, highlighting its better generalization capability.

3.3 Further Analysis

Effectiveness of Performance Prediction. We conduct ablation studies to evaluate the effectiveness of our dual-component representation for model-strategy pairs. We compare three configurations: (1) textual descriptions encoded using a sentence encoder, (2) randomly initialized learnable embeddings, and (3) a combination of both. As shown in Figure 5, the combined representation yields the highest accuracy in predicting model-strategy correctness, demonstrating the complementary strengths of prior knowledge from textual descriptions and task-adaptive learned embeddings.

Effectiveness of Token Usage Prediction. We evaluate the performance of our token length prediction. As shown in Figure 6, although reasoning and non-reasoning models differ substantially in average output length, the predictor achieves approximately 80% accuracy for non-reasoning

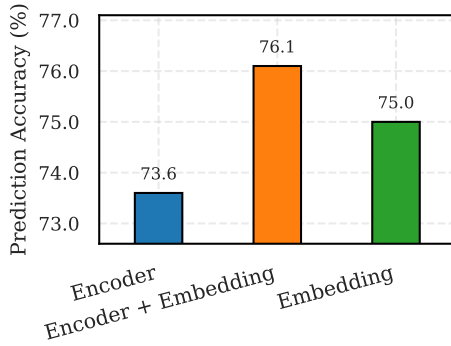


Figure 5: Distribution of the performance and average answer token of different LLMs in response to queries on the 4 reasoning tasks.

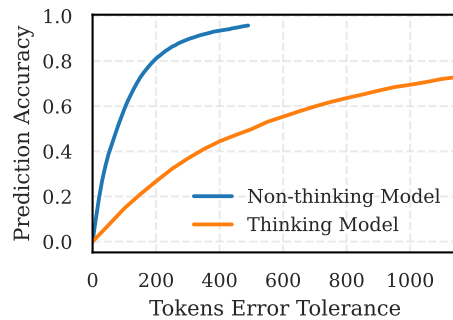


Figure 6: Distribution of the performance and average answer token of different reasoning strategies responding to queries on 4 reasoning tasks.

models within a 200-token error margin, and about 60% accuracy for reasoning models with a 600-token margin. Given that the output length of reasoning models can exceed 10,000 tokens, this level of coarse-grained prediction accuracy is sufficient for effective routing.

Different Routing Strategy. We further evaluate our model under various routing scenarios. We first consider routing based on a single fixed strategy. As shown in Figure 7, relying on a single strategy often leads to higher costs or suboptimal performance, as it cannot leverage the complementary strengths of different strategies. For example, strategies with greater reasoning depth may achieve higher accuracy but incur significantly higher costs. This underscores the advantage of dynamically selecting reasoning strategies.

We then examine three routing configurations by adjusting the controller parameters to reflect different trade-off preferences: *performance-first*, *cost-first*, and *balanced*, corresponding to RTR-performance, RTR-cost, and RTR-balanced in Figure 7, respectively. Experimental results show that our framework is flexible under different budget constraints and achieves competitive performance while adapting to varying priorities in cost-effectiveness and accuracy. Moreover, all three configurations outperform using only the vanilla strategy, highlighting the benefits of leveraging multiple reasoning strategies.

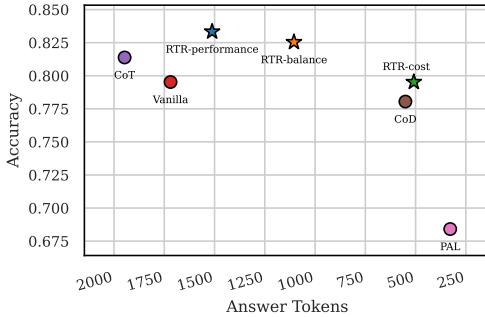


Figure 7: Distribution of different routing strategies. Routing with any of the four single strategies often results in suboptimal performance. All three routing configurations outperform the Vanilla baseline.

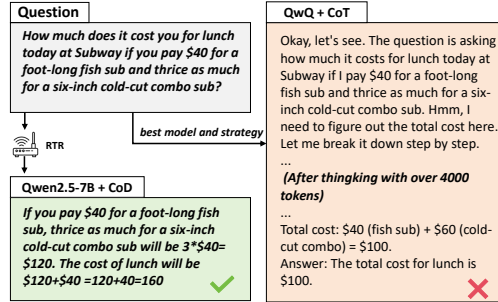


Figure 8: In this case, without using RTR, selecting the best model (QwQ) and the CoT strategy leads to redundant reasoning and ultimately an incorrect answer. In contrast, our RTR routes the query to Qwen2.5-7B with the CoD strategy and obtains the correct answer using only 32 tokens.

3.4 Case Study

Figure 8 illustrates a case showing the effectiveness of RTR. When RTR is disabled, the best-performing model (QwQ) along with the reasoning strategy of Chain-of-Thought results in redundant and unnecessarily verbose reasoning, ultimately leading to an incorrect answer. In contrast, when RTR is applied, it routes the query to Qwen2.5-7B and CoD. This configuration yields the correct answer while using only 32 tokens, demonstrating both improved accuracy and significantly reduced computation.

3.5 When To Think Routing

With the emergence of recent LLMs that support manual toggling between *thinking* and *non-thinking* modes, our framework is naturally compatible with such dual-mode models via binary routing. Specifically, we represent each mode as a distinct candidate in our model pool, this setup enables our router to automatically determine when to invoke the thinking mode, allowing for dynamic selection based on task requirements. To validate this capability, we conduct experiments with Qwen3-4B, which supports both reasoning and non-reasoning modes. As shown in Table 4, our framework effectively learns to trigger the reasoning mode only

Table 4: Performance of dynamic routing between reasoning and non-reasoning modes in Qwen3-4B.

Method	Acc (%)	Tokens
Qwen3-4B (non-thinking)	73.4	592.1
Qwen3-4B (thinking)	82.4	3112.8
Random	76.6	1834.2
RouteLLM	82.8	2247.1
KNN-Router	80.6	1418.7
EmbedLLM	83.2	2623.5
RTR (ours)	83.8	1321.1

when beneficial, thereby significantly reducing average token usage while maintaining high prediction accuracy. Moreover, dynamic strategy selection improves performance beyond that of any single-model baseline.

4 Related Work

Efficient LLM Reasoning. Recently, LLMs [3, 4, 43] have demonstrated increasingly powerful reasoning capabilities, but this progress has come at the expense of significantly higher inference-time computation. Recent studies have shown that such improvements often result in the problem of overthinking [14–21], where models tend to generate unnecessarily long and redundant reasoning steps, especially for simple queries. This has brought growing attention to the challenge of dynamically adapting inference based on problem difficulty. Existing approaches typically address this by fine-tuning [44, 45, 15] the model or applying reinforcement learning [14, 19, 20] to shorten reasoning trajectories and reduce unnecessary token generation, thereby enabling adaptive computation during inference.

Dynamic Reasoning Strategies in LLM. Chain-of-Thought (CoT) prompting [13] and its variants, such as decomposition-based methods [46, 9] and Tree-of-Thought (ToT) [47], have significantly improved performance across a wide range of reasoning tasks. Recent studies [34, 48, 35] have shown that these strategies exhibit complementary strengths depending on the characteristics of the task. For instance, while CoT performs well on mathematical and logical reasoning, it may not be universally effective for all question types. The Program-aided Language (PAL) approach [7] has shown superior performance on arithmetic-intensive tasks by incorporating code execution into the reasoning process. Recent work [34–36] has studied how to select the most suitable reasoning strategy for different types of questions.

LLM Routing. Model routing has emerged as one of the most cost-effective approaches in LLM ensembles [24, 31, 49, 28], as it selects a single model to generate responses for individual queries. RouteLLM [30] proposes four distinct strategies for routing between small and large models, effectively reducing cost. EmbedLLM [22] proposes learning universal model embeddings to facilitate routing decisions across diverse models. RouterBench [28] introduces a benchmark dataset for routing tasks and implements a range of routing baselines to balance response quality and computational cost. GraphRouter [24] adopts a graph-based approach by representing queries, tasks, and models as nodes in a heterogeneous graph to jointly predict model performance and cost. Recent works [25–27] have further explored collaborative multi-model deployments and the development of general-purpose routing systems. However, most existing approaches fail to consider the interplay between reasoning strategies and model performance. Our work addresses this gap by enabling more efficient inference and improved performance through joint reasoning strategy and model selection.

5 Discussion

Conclusion. In this paper, we present **RTR**, a unified framework for jointly selecting the optimal model and reasoning strategy for each query. To the best of our knowledge, RTR is the first approach to simultaneously address both model and strategy selection in a single routing framework. By representing all candidate models and strategies as learnable embeddings, our framework predicts two key metrics for each query: the expected performance score and the answer token usage across all available model-strategy pairs. These predictions are used to construct a routing table, enabling RTR to dynamically determine the best combination of model and reasoning strategy for each query. Extensive experiments on four in-distribution and three out-of distribution datasets demonstrate that RTR consistently makes effective routing decisions, achieving a favorable trade-off between overall performance and computational cost across all of the baselines. Furthermore, RTR is capable of dynamically deciding when to invoke the reasoning mode in models with binary thinking capabilities.

Limitations and Future Directions. Our current evaluation focuses on single-model selection, although incorporating collaborative decision-making across multiple models represents a promising avenue for future work. Additionally, while our experiments are centered on reasoning tasks, extending evaluation to a broader range of problem domains would further validate the framework’s

generality. We believe that efficient and intelligent routing systems are crucial for the future of LLMs, particularly in multi-model and resource-constrained scenarios.

References

- [1] K. Zhang, J. Li, G. Li, X. Shi, and Z. Jin, “Codeagent: Enhancing code generation with tool-integrated agent systems for real-world repo-level coding challenges,” 2024. [Online]. Available: <https://arxiv.org/abs/2401.07339>
- [2] J. Achiam, S. Adler, S. Agarwal, L. Ahmad, I. Akkaya, F. L. Aleman, D. Almeida, J. Altenschmidt, S. Altman, S. Anadkat *et al.*, “Gpt-4 technical report,” *arXiv preprint arXiv:2303.08774*, 2023.
- [3] A. Yang, B. Yang, B. Zhang, b. Hui, B. Zheng, B. Yu, C. Li, D. Liu, F. Huang, H. Wei *et al.*, “Qwen2. 5 technical report,” *arXiv preprint arXiv:2412.15115*, 2024.
- [4] D. Guo, D. Yang, H. Zhang, J. Song, R. Zhang, R. Xu, Q. Zhu, S. Ma, P. Wang, X. Bi *et al.*, “Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning,” *arXiv preprint arXiv:2501.12948*, 2025.
- [5] X. Wang, J. Wei, D. Schuurmans, Q. V. Le, E. H. Chi, S. Narang, A. Chowdhery, and D. Zhou, “Self-consistency improves chain of thought reasoning in language models,” in *The Eleventh International Conference on Learning Representations*, 2022.
- [6] M. Besta, N. Blach, A. Kubicek, R. Gerstenberger, M. Podstawski, L. Gianinazzi, J. Gajda, T. Lehmann, H. Niewiadomski, P. Nyczyk *et al.*, “Graph of thoughts: Solving elaborate problems with large language models,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 38, no. 16, 2024, pp. 17 682–17 690.
- [7] L. Gao, A. Madaan, S. Zhou, U. Alon, P. Liu, Y. Yang, J. Callan, and G. Neubig, “Pal: Program-aided language models,” in *International Conference on Machine Learning*. PMLR, 2023, pp. 10 764–10 799.
- [8] S. Xu, W. Xie, L. Zhao, and P. He, “Chain of draft: Thinking faster by writing less,” *arXiv preprint arXiv:2502.18600*, 2025.
- [9] T. Khot, H. Trivedi, M. Finlayson, Y. Fu, K. Richardson, P. Clark, and A. Sabharwal, “Decomposed prompting: A modular approach for solving complex tasks,” in *The Eleventh International Conference on Learning Representations*, 2022.
- [10] W. Chen, X. Ma, X. Wang, and W. W. Cohen, “Program of thoughts prompting: Disentangling computation from reasoning for numerical reasoning tasks,” *Transactions on Machine Learning Research*, 2023.
- [11] K. Cobbe, V. Kosaraju, M. Bavarian, M. Chen, H. Jun, L. Kaiser, M. Plappert, J. Tworek, J. Hilton, R. Nakano, C. Hesse, and J. Schulman, “Training verifiers to solve math word problems,” *arXiv preprint arXiv:2110.14168*, 2021.
- [12] OpenAI. (2024) Introducing gpt-4o and gpt-4o-mini. Accessed: 2025-05-16. [Online]. Available: <https://openai.com/index/introducing-o3-and-o4-mini/>
- [13] J. Wei, X. Wang, D. Schuurmans, M. Bosma, F. Xia, E. Chi, Q. V. Le, D. Zhou *et al.*, “Chain-of-thought prompting elicits reasoning in large language models,” *Advances in neural information processing systems*, vol. 35, pp. 24 824–24 837, 2022.
- [14] X. Chen, J. Xu, T. Liang, Z. He, J. Pang, D. Yu, L. Song, Q. Liu, M. Zhou, Z. Zhang *et al.*, “Do not think that much for 2+ 3=? on the overthinking of o1-like llms,” *arXiv preprint arXiv:2412.21187*, 2024.
- [15] R. Manvi, A. Singh, and S. Ermon, “Adaptive inference-time compute: LLMs can predict if they can do better, even mid-generation,” *arXiv preprint arXiv:2410.02725*, 2024.
- [16] M. Damani, I. Shenfeld, A. Peng, A. Bobu, and J. Andreas, “Learning how hard to think: Input-adaptive allocation of LM computation,” in *The Thirteenth International Conference on Learning Representations*, 2025.
- [17] Y. Liu, J. Wu, Y. He, H. Gao, H. Chen, B. Bi, J. Zhang, Z. Huang, and B. Hooi, “Efficient inference for large reasoning models: A survey,” *arXiv preprint arXiv:2503.23077*, 2025.
- [18] Z.-Z. Li, D. Zhang, M.-L. Zhang, J. Zhang, Z. Liu, Y. Yao, H. Xu, J. Zheng, P.-J. Wang, X. Chen *et al.*, “From system 1 to system 2: A survey of reasoning large language models,” *arXiv preprint arXiv:2502.17419*, 2025.

- [19] Y. Sui, Y.-N. Chuang, G. Wang, J. Zhang, T. Zhang, J. Yuan, H. Liu, A. Wen, H. Chen, X. Hu *et al.*, “Stop overthinking: A survey on efficient reasoning for large language models,” *arXiv preprint arXiv:2503.16419*, 2025.
- [20] W. Yang, S. Ma, Y. Lin, and F. Wei, “Towards thinking-optimal scaling of test-time compute for llm reasoning,” *arXiv preprint arXiv:2502.18080*, 2025.
- [21] Q. Chen, L. Qin, J. Liu, D. Peng, J. Guan, P. Wang, M. Hu, Y. Zhou, T. Gao, and W. Che, “Towards reasoning era: A survey of long chain-of-thought for reasoning large language models,” *arXiv preprint arXiv:2503.09567*, 2025.
- [22] R. Zhuang, T. Wu, Z. Wen, A. Li, J. Jiao, and K. Ramchandran, “EmbedLLM: Learning compact representations of large language models,” in *The Thirteenth International Conference on Learning Representations*, 2025.
- [23] S. Chen, W. Jiang, B. Lin, J. Kwok, and Y. Zhang, “Routerdc: Query-based router by dual contrastive learning for assembling large language models,” *Advances in Neural Information Processing Systems*, vol. 37, pp. 66 305–66 328, 2024.
- [24] T. Feng, Y. Shen, and J. You, “Graphrouter: A graph-based router for LLM selections,” in *The Thirteenth International Conference on Learning Representations*, 2025.
- [25] K. Mei, W. Xu, S. Lin, and Y. Zhang, “Eccos: Efficient capability and cost coordinated scheduling for multi-llm serving,” *Available at SSRN 5159339*, 2025.
- [26] X. Wang, Y. Liu, W. Cheng, X. Zhao, Z. Chen, W. Yu, Y. Fu, and H. Chen, “Mixllm: Dynamic routing in mixed large language models,” *arXiv preprint arXiv:2502.18482*, 2025.
- [27] W. Jitkritum, H. Narasimhan, A. S. Rawat, J. Juneja, Z. Wang, C.-Y. Lee, P. Shenoy, R. Panigrahy, A. K. Menon, and S. Kumar, “Universal model routing for efficient llm inference,” *arXiv preprint arXiv:2502.08773*, 2025.
- [28] Q. J. Hu, J. Bieker, X. Li, N. Jiang, B. Keigwin, G. Ranganath, K. Keutzer, and S. K. Upadhyay, “Routerbench: A benchmark for multi-LLM routing system,” in *Agentic Markets Workshop at ICML 2024*, 2024.
- [29] K. Lu, H. Yuan, R. Lin, J. Lin, Z. Yuan, C. Zhou, and J. Zhou, “Routing to the expert: Efficient reward-guided ensemble of large language models,” in *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, 2024, pp. 1964–1974.
- [30] I. Ong, A. Almahairi, V. Wu, W.-L. Chiang, T. Wu, J. E. Gonzalez, M. W. Kadous, and I. Stoica, “RouteLLM: Learning to route LLMs from preference data,” in *The Thirteenth International Conference on Learning Representations*, 2025.
- [31] Z. Chen, J. Li, P. Chen, Z. Li, K. Sun, Y. Luo, Q. Mao, D. Yang, H. Sun, and P. S. Yu, “Harnessing multiple large language models: A survey on llm ensemble,” *arXiv preprint arXiv:2502.18036*, 2025.
- [32] X. Xu, Y. Xu, T. Chen, Y. Yan, C. Liu, Z. Chen, Y. Wang, Y. Yin, Y. Wang, L. Shang *et al.*, “Teaching llms according to their aptitude: Adaptive reasoning for mathematical problem solving,” *arXiv preprint arXiv:2502.12022*, 2025.
- [33] X. Wang, S. Feng, Y. Li, P. Yuan, Y. Zhang, C. Tan, B. Pan, Y. Hu, and K. Li, “Make every penny count: Difficulty-adaptive self-consistency for cost-efficient reasoning,” *arXiv preprint arXiv:2408.13457*, 2024.
- [34] M. Yue, W. Yao, H. Mi, D. Yu, Z. Yao, and D. Yu, “Dots: Learning to reason dynamically in llms via optimal reasoning trajectories search,” *arXiv preprint arXiv:2410.03864*, 2024.
- [35] T. Parekh, P. Prakash, A. Radovic, A. Shekher, and D. Savenkov, “Dynamic strategy planning for efficient question answering with large language models,” *arXiv preprint arXiv:2410.23511*, 2024.
- [36] S. A. Aytes, J. Baek, and S. J. Hwang, “Sketch-of-thought: Efficient llm reasoning with adaptive cognitive-inspired sketching,” *arXiv preprint arXiv:2503.05179*, 2025.
- [37] D. Hendrycks, C. Burns, S. Kadavath, A. Arora, S. Basart, E. Tang, D. Song, and J. Steinhardt, “Measuring mathematical problem solving with the math dataset,” *arXiv preprint arXiv:2103.03874*, 2021.
- [38] C. He, R. Luo, Y. Bai, S. Hu, Z. L. Thai, J. Shen, J. Hu, X. Han, Y. Huang, Y. Zhang *et al.*, “Olympiadbench: A challenging benchmark for promoting agi with olympiad-level bilingual multimodal scientific problems,” *arXiv preprint arXiv:2402.14008*, 2024.

- [39] D. Hendrycks, C. Burns, S. Basart, A. Zou, M. Mazeika, D. Song, and J. Steinhardt, “Measuring massive multitask language understanding,” *Proceedings of the International Conference on Learning Representations (ICLR)*, 2021.
- [40] J. Welbl, N. F. Liu, and M. Gardner, “Crowdsourcing multiple choice science questions,” *arXiv preprint arXiv:1707.06209*, 2017.
- [41] Y. Bisk, R. Zellers, J. Gao, Y. Choi *et al.*, “Piqa: Reasoning about physical commonsense in natural language,” in *Proceedings of the AAAI conference on artificial intelligence*, vol. 34, no. 05, 2020, pp. 7432–7439.
- [42] P. Clark, I. Cowhey, O. Etzioni, T. Khot, A. Sabharwal, C. Schoenick, and O. Tafjord, “Think you have solved question answering? try arc, the ai2 reasoning challenge,” *arXiv:1803.05457v1*, 2018.
- [43] A. Hurst, A. Lerer, A. P. Goucher, A. Perelman, A. Ramesh, A. Clark, A. Ostrow, A. Welihinda, A. Hayes, A. Radford *et al.*, “Gpt-4o system card,” *arXiv preprint arXiv:2410.21276*, 2024.
- [44] J. Pan, Y. Zhang, C. Zhang, Z. Liu, H. Wang, and H. Li, “Dynathink: Fast or slow? a dynamic decision-making framework for large language models,” *arXiv preprint arXiv:2407.01009*, 2024.
- [45] Y. Kang, X. Sun, L. Chen, and W. Zou, “C3ot: Generating shorter chain-of-thought without compromising effectiveness,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 39, no. 23, 2025, pp. 24 312–24 320.
- [46] Y. Zhao, T. Ji, W. Feng, Z. Huang, Q. Liu, Z. Liu, Y. Ma, K. Zhang, and E. Chen, “Unveiling the magic of code reasoning through hypothesis decomposition and amendment,” *arXiv preprint arXiv:2502.13170*, 2025.
- [47] S. Yao, D. Yu, J. Zhao, I. Shafran, T. Griffiths, Y. Cao, and K. Narasimhan, “Tree of thoughts: Deliberate problem solving with large language models,” *Advances in Neural Information Processing Systems*, vol. 36, 2024.
- [48] Z. R. Sprague, F. Yin, J. D. Rodriguez, D. Jiang, M. Wadhwa, P. Singhal, X. Zhao, X. Ye, K. Mahowald, and G. Durrett, “To cot or not to cot? chain-of-thought helps mainly on math and symbolic reasoning,” in *The Thirteenth International Conference on Learning Representations*, 2025.
- [49] D. Ding, A. Mallick, C. Wang, R. Sim, S. Mukherjee, V. Rühle, L. V. S. Lakshmanan, and A. H. Awadallah, “Hybrid LLM: Cost-efficient and quality-aware query routing,” in *The Twelfth International Conference on Learning Representations*, 2024.

A Extended Experiment Results

A.1 Effectiveness of Multi-Strategy and Multi-Model Selection

Effectiveness of Multi-Strategy. We evaluate the benefits of multi-strategy selection by comparing the original accuracy and the routing accuracy of our proposed RTR to those of individual models. As shown in Figure 9, we compare using RTR within a single model (labeled as ‘Model + RTR’, marked by $\times$) to the model’s original overall accuracy (labeled as ‘Model Overall’, marked by $\circ$). Routing across multiple strategies consistently improves performance across all models, highlighting the value of strategy selection tailored to instance characteristics.

Effectiveness of Multi-Model. As shown in Figure 9, we plot the λ -curve (λ controls the trade-off between token usage and performance) to evaluate the effectiveness of multi-model selection. While RTR’s λ -curve shows only a slight performance drop compared to routing on QwQ alone, it significantly outperforms most individual models. This demonstrates that leveraging multiple models with strategy-aware routing effectively balances accuracy and efficiency.

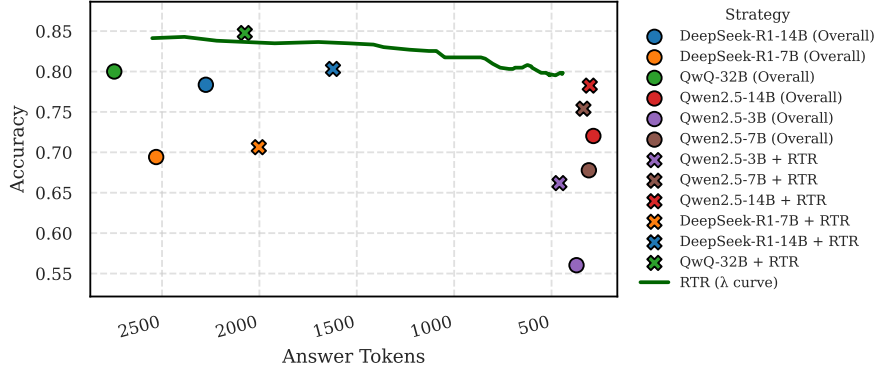


Figure 9: Effectiveness of Multi-Model and Multi-Strategy.

A.2 Effectiveness of Training Samples

We evaluate the prediction performance of the performance predictor under varying training sample sizes. As shown in Figure 10, an increase in the number of training samples leads to a general improvement in prediction accuracy across all datasets. This enhancement is particularly pronounced when the training sample size is relatively small. As the training size approaches around 5000 samples, the performance begins to converge. This suggests that our prediction framework is capable of effectively modeling the relationship between model-strategy pairs and problems using a relatively small amount of data, thereby enabling accurate performance prediction.

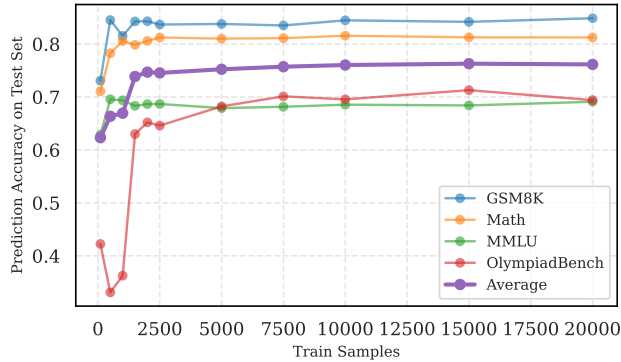


Figure 10: Prediction accuracy on the test set under different training sample sizes.

A.3 More Case Studies

We show several cases of RTR in Figures 11, 12, and 13 to illustrate its ability to adaptively select model-strategy pairs. As demonstrated in Figure 11, when faced with relatively simple problems, RTR selects a non-reasoning model and a lightweight strategy, which also leads to a correct answer with a lower token cost. In Figures 12 and 13, when handling harder questions, RTR benefits from accurate performance and token cost prediction, ensuring both correctness and lower token usage. This demonstrates the effectiveness of RTR in model-strategy pair routing.

Routing Table							
Question	Model	Strategy	Pred Perf	Pred Ans Length	Route Score	GT Label	GT Ans Length
Raymond and Samantha are cousins. Raymond was born 6 years before Samantha. Raymond had a son at the age of 23. If Samantha is now 31, how many years ago was Raymond's son born? (Ground Truth: 14)	Qwen2.5-14B	CoT	0.982	275.9	0.975	True	170.0
	QwQ-32B	CoD	0.986	568.6	0.972	True	392.0
	QwQ-32B	CoT	0.991	1131.5	0.963	True	635.0
	DeepSeek-R1-14B	CoD	0.961	320.1	0.953	True	217.0
	DeepSeek-R1-14B	CoT	0.971	789.2	0.951	True	290.0
	DeepSeek-R1-14B	Vanilla	0.969	771.6	0.950	True	310.0
	Qwen2.5-7B	CoT	0.948	310.9	0.940	False	229.0
	QwQ-32B	Vanilla	0.979	1607.1	0.939	True	923.0
	Qwen2.5-14B	PAL	0.927	209.2	0.921	True	157.0
	Qwen2.5-3B	CoT	0.930	375.7	0.920	False	191.0
	DeepSeek-R1-7B	CoT	0.946	1023.7	0.920	False	378.0
	DeepSeek-R1-7B	Vanilla	0.945	1006.9	0.919	False	290.0
	DeepSeek-R1-7B	CoD	0.917	299.3	0.910	False	144.0
...							

Question (GSM8K):
Raymond and Samantha are cousins. Raymond was born 6 years before Samantha. Raymond had a son at the age of 23. If Samantha is now 31, how many years ago was Raymond's son born? (**Ground Truth:** 14)

Fixed Best $\rightarrow$ QwQ-32B + CoT :
<think> Okay, let's see. The problem is about figuring out how many years ago Raymond's son was born, given some information about their ages. Let me break it down step by step. First, the question states that Raymond and Samantha are cousins, and Raymond was born 6 years before Samantha. So, Raymond is older than Samantha by 6 years. Currently, Samantha is 31 years old. That means Raymond's current age should be Samantha's age plus those 6 years. Let me check that again. Wait, if Raymond was born 6 years before Samantha, then yes, he is 6 years older. So if Samantha is 31 now, Raymond must be $31 + 6 = 37$ years old right now. Next part: Raymond had a son at the age of 23. So, we need to find out how many years ago that was. Let's think. If Raymond is currently 37, and he was 23 when his son was born, then the number of years that have passed since his son was born is $37 - 23 = 14$. Wait, that would mean his son is 14 years old now. So the son was born 14 years ago. Hmm, but let me make sure I didn't mix up anything. Wait, let me rephrase all the steps to confirm: (**More Thinking Steps ...**) </think> Raymond is 6 years older than Samantha. Since Samantha is currently 31, Raymond is $31 + 6 = 37$ years old. Raymond had his son at age 23, so the number of years since his son's birth is $37 - 23 = 14$. The answer is 14. (**Total 635 tokens**)

RTR $\xrightarrow{\text{route}}$ Qwen2.5-14B + CoT :
To solve this problem, we need to determine the current age of Raymond and then figure out how long ago his son was born. First, determine Raymond's current age. Samantha is currently 31 years old. Raymond was born 6 years before Samantha. Therefore, Raymond is currently $31 + 6 = 37$ years old. Next, determine when Raymond's son was born. Raymond had a son when he was 23 years old. Raymond is currently 37 years old. The difference between Raymond's current age and the age when he had his son is $37 - 23 = 14$ years. Therefore, Raymond's son was born 14 years ago. 14. (**Total 170 tokens**)

Figure 11: An easy case from GSM8K. The fixed-best model-strategy pair incurs a token cost of 635. RTR predicts a high cost for this option and instead routes the question to a lightweight non-reasoning model (Qwen2.5-14B), which answers correctly using only 170 tokens.

Routing Table							
Question	Model	Strategy	Pred Perf	Pred Ans Length	Route Score	GT Label	GT Ans Length
Divide the reciprocal of $\frac{5}{6}$ by the reciprocal of $\frac{5}{6}$.	DeepSeek-R1-14B	CoD	0.979	619.3	0.964	True	216.0
	QwQ-32B	CoD	0.994	1414.0	0.959	True	1756.0
	Qwen2.5-14B	CoT	0.967	453.3	0.956	True	333.0
	QwQ-32B	CoT	0.995	2051.1	0.943	True	1963.0
	DeepSeek-R1-14B	CoT	0.980	1815.3	0.934	True	3387.0
	DeepSeek-R1-7B	CoD	0.946	720.0	0.928	True	196.0
	DeepSeek-R1-14B	Vanilla	0.976	1978.9	0.927	True	2273.0
	QwQ-32B	Vanilla	0.988	2726.9	0.920	True	2834.0
	Qwen2.5-7B	CoT	0.922	468.6	0.910	True	400.0
	DeepSeek-R1-7B	CoT	0.957	2367.1	0.898	False	17000.0
	DeepSeek-R1-7B	Vanilla	0.954	2493.2	0.892	True	1972.0
...							
Question (Math) : Divide the reciprocal of $\frac{5}{6}$ by the reciprocal of the reciprocal of $\frac{5}{6}$. (Ground Truth: $\frac{36}{25}$) Fixed Best $\rightarrow$ QwQ-32B + CoT : $\langle \text{think} \rangle$ Okay, let's tackle this problem step by step. The question says: "Divide the reciprocal of $5/6$ by the reciprocal of the reciprocal of $5/6$." Hmm, that sounds a bit convoluted, but let's break it down. First, I need to recall what a reciprocal is. The reciprocal of a fraction is just flipping the numerator and denominator. So, the reciprocal of $5/6$ would be $6/5$. Got that part. Now, the problem mentions "the reciprocal of the reciprocal of $5/6$." Let me parse that. The first reciprocal of $5/6$ is $6/5$. Then the reciprocal of that would be flipping $6/5$ back to $5/6$. So, the reciprocal of the reciprocal of $5/6$ is just $5/6$ again. Wait, so the second part simplifies back to the original fraction? That makes sense because taking the reciprocal twice cancels out the operation. So now the problem is asking me to divide the first reciprocal (which is $6/5$) by the second term, which we just found out is $5/6$. So mathematically, (More Thinking Steps ...) $\langle \text{think} \rangle$ First, we need to find the reciprocal of $\frac{5}{6}$, which is $\frac{6}{5}$. Next, we determine the reciprocal of the reciprocal of $\frac{5}{6}$. The reciprocal of $\frac{6}{5}$ is $\frac{5}{6}$. We then need to divide the first result by the second result. This means we need to compute $\frac{6}{5} \div \frac{5}{6}$. To divide by a fraction, we multiply by its reciprocal: $\frac{6}{5} \div \frac{5}{6} = \frac{6}{5} \times \frac{6}{5} = \frac{6 \times 6}{5 \times 5} = \frac{36}{25}$ Thus, the final answer is $\boxed{\frac{36}{25}}$. (Total 1963 tokens) RTR $\xrightarrow{\text{route}}$ DeepSeek-R1-14B + CoD : $\langle \text{think} \rangle$ First, I need to find the reciprocal of $5/6$, which is $6/5$. Next, I determine the reciprocal of the reciprocal of $5/6$. The reciprocal of $5/6$ is $6/5$, and the reciprocal of that is $5/6$. Finally, I divide the first reciprocal ($6/5$) by the second reciprocal ($5/6$), which simplifies to $(6/5) * (6/5) = 36/25$. $\langle \text{think} \rangle$ Step-by-step minimal draft: 1. Reciprocal of $5/6$: $6/5$ 2. Reciprocal of reciprocal: $5/6$ 3. Divide: $(6/5) \div (5/6)$ 4. Multiply by reciprocal: $(6/5) \times (6/5)$ 5. Result: $36/25$ The Answer is $\boxed{\frac{36}{25}}$. (216 tokens)							

Figure 12: A case from Math. The fixed-best model-strategy pair answers the question with a token cost of 1963. RTR instead selects DeepSeek-R1-14B with Chain-of-Thought (CoD), which correctly solves the problem using only 216 tokens. Since the question benefits from a step-by-step draft-style reasoning process, this pair provides a clear and efficient solution at a much lower token cost.

Routing Table							
Question	Model	Strategy	Pred Perf	Pred Ans Length	Route Score	GT Label	GT Ans Length
For any two different real numbers x and y ...	QwQ-32B	CoD	0.556	9756.9	0.312	True	9157.0
	Qwen2.5-14B	CoT	0.322	898.0	0.300	False	599.0
	Qwen2.5-14B	PAL	0.308	580.2	0.294	False	678.0
	QwQ-32B	CoT	0.569	11623.6	0.278	True	11767.0
	Qwen2.5-14B	CoD	0.273	190.8	0.268	False	90.0
	Qwen2.5-14B	Vanilla	0.280	935.0	0.257	False	518.0
	Qwen2.5-7B	Vanilla	0.246	915.2	0.223	False	695.0
	QwQ-32B	Vanilla	0.519	12125.3	0.215	True	11671.0
	Qwen2.5-7B	CoD	0.209	256.5	0.202	False	73.0
	DeepSeek-R1-14B	CoD	0.391	10674.2	0.124	True	3422.0
	Qwen2.5-3B	CoD	0.113	384.0	0.104	False	170.0
	Qwen2.5-3B	CoT	0.131	1119.9	0.103	False	621.0
	DeepSeek-R1-7B	CoD	0.289	9690.2	0.046	False	3762.0
	DeepSeek-R1-7B	CoT	0.342	12369.1	0.033	False	17000.0
	DeepSeek-R1-14B	CoT	0.336	13275.0	0.004	False	20481.0
	DeepSeek-R1-7B	Vanilla	0.308	12749.7	-0.011	False	17000.0
	DeepSeek-R1-14B	Vanilla	0.320	13563.4	-0.019	False	20481.0

🔍 *Question (OlympiadBench)* :

For any two different real numbers x and y , we define $D(x, y)$ to be the unique integer d satisfying $2^d \leq |x - y| < 2^{d+1}$. Given a set of reals $\mathcal{F}$, and an element $x \in \mathcal{F}$, we say that the scales of x in $\mathcal{F}$ are the values of $D(x, y)$ for $y \in \mathcal{F}$ with $x \neq y$. Let k be a given positive integer. Suppose that each member x of $\mathcal{F}$ has at most k different scales in $\mathcal{F}$ (note that these scales may depend on x). What is the maximum possible size of $\mathcal{F}$? (**Ground Truth:** 2^k)

⚙️ *Fixed Best $\rightarrow$ QwQ-32B + CoT* :

<think>Okay, so I need to figure out the maximum possible size of a set F where each element in F has at most k different scales. The scales here are defined as the values of $D(x, y)$ for each pair of distinct elements x and y in F . And $D(x, y)$ is the unique integer d such that $2^d \leq |x - y| < 2^{d+1}$. Hmm, let me start by understanding the problem step by step. (**More Thinking Steps ...**) </think>

To solve the problem, we need to determine the maximum possible size of a set $\mathcal{F}$ such that each element in $\mathcal{F}$ has at most k different scales. The scales are defined by the function $D(x, y)$, which categorizes the distance between any two elements x and y into intervals of powers of two. (**More Reasoning Steps ...**)

The recursive doubling of the set size for each increment in k leads to the conclusion that the maximum size of $\mathcal{F}$ is 2^k . The maximum possible size of the set $\mathcal{F}$ is 2^k . (**11767 tokens**)

🦋 *RTR $\xrightarrow{\text{route}}$ QwQ-32B + CoD* :

<think>Okay, so I need to figure out the maximum possible size of a set F where each element has at most k different scales. The scales are defined as $D(x, y)$, which is the integer d such that $2^d \leq |x - y| < 2^{d+1}$. Each element x in F can have at most k different $D(x, y)$ values when compared to all other elements in F . Hmm, let's start by understanding the problem better. (**More Thinking Steps ...**) </think>

1. Define scales via $D(x, y) = d$ where $2^d \leq |x - y| < 2^{d+1}$.
2. Each element x has at most k different $D(x, y)$ values.
3. For maximum size, arrange points such that pairwise distances fall into k scales.
4. Construct the set $\{0, 1, 2, \dots, 2^k - 1\}$: each element's distances to others fall into scales 0 to $k - 1$.
5. This gives exactly k scales per element, achieving 2^k elements.

The Answer is 2^k (**9157 tokens**)

Figure 13: A hard question from OlympiadBench. As shown in the routing table, most model-strategy pairs fail to answer this question. RTR accurately selects the strongest model, QwQ-32B, along with the CoD strategy, which reduces the token cost while preserving answer correctness. This leads to a correct answer with a token reduction of over 2500.

B Additional Details on Methodology and Experiments

B.1 Baseline Implementation Details

We describe the implementation details of the baseline methods as follows. For the **Random** router, a model-strategy pair is randomly selected for each input. We report the average performance over 50 runs. For the **KNN-Router**, we set the parameter k to 10. For each model-strategy pair, we compute its performance over all neighboring queries, where the score is defined as the average accuracy minus the average output length multiplied by a parameter λ . We tune λ to balance cost and performance, and report the best result. For **RouteLLM**, since this is a binary router, we label each query according to its average accuracy on the candidate models, and train a binary classifier to decide which model to route each query to. We tune the balance parameter and report the best result. The strong and weak models used are QwQ and Qwen2.5-7B, respectively. For **EmbedLLM**, we assign a unique ID to each model-strategy pair and learn a 768-dimensional embedding for each pair. Following the original method, we frame the learning process as a reconstruction task: given a matrix of model correctness across prompts, a reconstruction network is trained to recover this matrix, thereby enforcing the embeddings to capture the behavioral characteristics of each model-strategy pair.

B.2 Prompts Used in Experiments

CoT

Please reason step by step before providing the final answer, and put your final answer within `\\boxed{\\{\\}}`.

CoD

Think step by step, but only keep minimum draft for each thinking step, with 5 words at most, and put your final answer within `\\boxed{\\{\\}}`.

PAL

Write a Python code snippet to solve the following problem. Do not use any plotting libraries or the `input()` function.

B.3 Profiles of Strategies and Models

Vanilla: Vanilla prompting retains the original question content without adding any additional prompt information.

Chain-of-Thought (CoT): Chain-of-Thought (CoT) prompting guides the model to articulate a step-by-step reasoning process before providing the final answer. This results in longer responses and slower reasoning speed, typically generating the longest answers, but it performs best on complex problems such as mathematical reasoning.

Chain-of-Draft (CoD): Chain-of-Draft (CoD) prompts the model to generate only intermediate drafts with explicit constraints on output length, encouraging concise reasoning. These drafts represent the model’s thinking process, often containing important calculation steps and key reasoning information. It simplifies the intermediate steps of the reasoning chain while retaining good performance, resulting in shorter answers.

Program-Aided Language (PAL): Program-Aided Language (PAL) transforms the reasoning process into executable code. This approach leverages the determinism of programming languages to ensure logical consistency and high accuracy, making it particularly effective for mathematical, symbolic, or algorithmic tasks. PAL relies on a suitable code execution environment and consistently produces results with high reliability and stable, moderately sized outputs. However, it may not be well suited for commonsense reasoning tasks.

Qwen2.5-3B-Instruct: Qwen2.5-3B-Instruct is a lightweight 3B parameter model with fast inference and low resource usage. It is suitable for simple tasks such as basic question answering and short-form text generation, but is limited in handling complex reasoning or multi-step tasks.

Qwen2.5-7B-Instruct: Qwen2.5-7B-Instruct is a mid-small 7B parameter model that balances speed and performance. It is capable of multi-turn dialogue, basic code and math tasks, and offers improved language understanding over smaller models, while maintaining efficient inference.

Qwen2.5-14B-Instruct: Qwen2.5-14B-Instruct is a mid-sized 14B parameter model that excels at complex reasoning, document summarization, and structured mid-length text generation. It demonstrates strong performance on tasks requiring deeper understanding and context retention.

DeepSeek-R1-7B: DeepSeek-R1-7B is a 7B distilled model with slightly slower but stable inference compared to other models of similar size. It is well-suited for medium-length answers that require deep reasoning, and often generates more detailed and comprehensive responses.

DeepSeek-R1-14B: DeepSeek-R1-14B is a 14B distilled model with slower inference but strong logical and mathematical capabilities. It is ideal for mathematical proofs and tasks requiring rigorous step-by-step reasoning, offering robust performance in logic-intensive scenarios.

QwQ-32B: QwQ-32B is a large 32B quantized model with slow inference. It excels at complex logic, coding, and multi-step reasoning tasks, though it may produce verbose outputs. Its large capacity enables handling of challenging prompts and long-context tasks.

Qwen3-4B (thinking): Qwen3-4B in 'thinking' mode generates longer reasoning chains and detailed thought processes. While it has slower inference and higher resource usage, it excels at solving complex logic and reasoning problems, making it suitable for tasks that require step-by-step explanations or in-depth analysis.

Qwen3-4B (non-thinking): Qwen3-4B in 'non-thinking' mode is optimized for short, direct answers. It provides fast inference with low resource cost, but is limited in deep reasoning or step-by-step explanations, making it best for straightforward queries or when efficiency is prioritized.